SearchWiki: Learning to Build and Navigate Knowledge Wikis for Active Information Seeking

Guransh Singh[1*], **Vishwajeet Kumar**[2], **Arkadeep Acharya**[2], **Adnan Qidwai**[2], **Jaydeep Sen**[2], **Sachindra Joshi**[2]

[1]Birla Institute of Technology, Mesra
[2]IBM
guransh766@gmail.com, vishk024@in.ibm.com, acharyarka17@ibm.com, Adnan.Qidwai1@ibm.com, jaydesen@in.ibm.com, jsachind@in.ibm.com

arXiv:2608.29953v1 [cs.AI] 30 Aug 2026

Abstract

Flat retrieval-augmented generation treats a corpus as a bag of chunks, discarding document hierarchy and cross document structure. We introduce **SearchWiki**, a harness framework that synthesizes a corpus into a hierarchical, typed, navigable wiki and trains an agent, **WikiResearcher-9B**, to retrieve information through multi-turn tool use. The wiki organizes knowledge into three layers - document overviews, cross-document topic pages, and page-level source records; enabling progressive refinement of retrieval when initial lookup misses. We optimize the agent's navigation policy with on-policy reinforcement learning with a multi-component reward function balancing answer correctness, retrieval quality and trajectory efficiency. Evaluation on ViDoRe-V3 (8 domains), FinanceBench, and memory benchmarks (LoCoMo, LongMemEval, PersonaMem-v2) shows that **WikiResearcher-9B** which is our RL-tuned Qwen 9B model, significantly outperforms same-size untrained baselines and exceeds or matches larger external models. **SearchWiki** paired with **WikiResearcher-9B** demonstrates that learned navigation over structured corpora is a superior alternative to flat retrieval.

1 Introduction

Retrieval augmented generation (RAG) has emerged as a dominant paradigm for grounding large language models on external knowledge (Lewis et al. 2020; Gao et al. 2023), mitigating hallucination (Shuster et al. 2021) and enabling access to information beyond model's parametric memory or training cutoff. In its canonical form, RAG operates by embedding a corpus into a vector space, retrieving the top-k chunks most similar to a query (Karpukhin et al. 2020), and conditioning generation on this retrieved context - an approach that has proven remarkably effective for open domain question answering(QA), enterprise search, and knowledge intensive/grounded NLP tasks more broadly. However, the "flat" formulation of RAG treats corpus as an undifferentiated bag of chunks, indexed and retrieved independently of one another. In doing so, it discards the very structure that makes the corpus navigable: the hierarchy of chapter, sections, sub-sections and the cross references and the citations linking the documents and the implicit organization that a human reader relies on to build context incrementally (Liu et al. 2021; Sarthi et al. 2024; Edge et al. 2024; Gutiérrez et al. 2024). Critically, flat RAG offers no recovery mechanism when initial retrieval fails - if the top-k chunks do not contain the answer, there is no mechanism for the system to reformulate its search, backtrack, or explore related regions of the corpus, since retrieval is a single-shot operation than an iterative process. The agent, in effect cannot navigate; it can only query and hope. As the corpora grow larger and more complex spanning technical documentations, legal codices, financial reports or scientific literature with deep hierarchical dependencies, answering non-trivial questions increasingly requires multi-step, structured reasoning that flat RAG/retrieval architectures are fundamentally unequipped to provide (Trivedi et al. 2023; Jiang et al. 2023; Asai et al. 2024; Jeong et al. 2024; Yao et al. 2023).

Consider how a human researcher approaches an unfamiliar body of knowledge: not by scanning every paragraph in isolation, but by navigating hierarchically starting from an overview of table of contents, drilling down into relevant chapters, sections, subsections, concepts and following cross references until sufficient detail is gathered to answer question at hand. This process is inherently structured and recoverable: if a chosen path proves unproductive, the reader backtracks to a higher level and tries another branch rather than abandoning the search entirely (Bates 1989; Pirolli and Card 1999). Flat RAG systems, by contrast have no notion of "levels" to retreat to or "branches" to explore, every chunk is equally proximate to every query, there is no scaffold on which the agent might orient itself. This raises a natural question: why doesn't RAG simply adopt the same navigational strategy as humans? Our insight is that this capability can be manufactured rather than assumed. Specifically, we propose to synthesize a wiki-like hierarchical structure directly from the unstructured corpus automatically by inducing overview pages, concept level summaries, cross document links (Ming et al. 2026; Dai et al. 2026; Du et al. 2026; Sun, Wei, and Hsieh 2026; Xu et al. 2026) and train an agent via reinforcement learning (RL), to navigate the synthesized structure much as a human would traverse a wiki. We argue that information seeking over corpora should move from **passive retrieval** to **active corpus navigation** (Hui et al. 2026; Salemi et al. 2026): the agent should choose what to inspect, in what order, and at what granularity, using intermediate observations to decide the next step.

[*]Work done as an intern at IBM

We instantiate this insight into two-stage stage framework called **SearchWiki**, First, *synthesis:* an LLM processes the raw corpus and builds a typed, three-layer wiki:

- **Document overviews (*documents/*):** per-document summaries and optional section subfolders that act as entry points and proxies to direct sources.
- **Topics (*topics/*):** cross-document concept pages with markdown links to related documents and topics, to bridge the cross-document and cross-section information gap.
- **Sources (*sources/*):** page-level JSON records (OCR/layout text and related metadata) for fine-grained verification and cited answering.

This layout supports *progressive disclosure*: the agent can start from summaries or topics and descend to specific pages only when needed. The same tool suite (*search_wiki*, *read_file*, *read_page*, *grep_search*, and related filesystem tools) operates over any wiki compiled into this convention. Second, *learning:* we train **WikiResearcher-9B**, an agent via RL to navigate this wiki, deciding when to ascend into details and when to backtrack, conditioned on the query. This works because it replaces passive, single-shot retrieval with a learned policy operating over explicit structure: the wiki provides a navigable scaffold, while RL training gives the agent the judgment to use it efficiently and recover from dead ends. Two design choices matter most: the three-layer hierarchy balances global orientation against local precision, and a multi-component reward jointly optimizes for answer correctness, navigation efficiency, and effective backtracking.

We train the navigation policy with on-policy RL using Decoupled Clip and Dynamic Sampling Policy Optimization (Yu et al. 2025) (DAPO), building on group-relative advantages introduced by GRPO (Shao et al. 2024). Rollouts are multi-turn tool trajectories over live wiki environments. **Retrieved Token Masking (RTM)** (Jin et al. 2025) restricts policy gradients to model-generated tokens (planning, tool calls, and answers), not tool observations.

Our contributions are threefold:

1. A domain-agnostic wiki compilation and navigation interface for any knowledge source, with progressive disclosure over document, topic, and source layers.
2. An online RL training recipe (DAPO, RTM, retrieval-aligned dense rewards) for tuning multi-turn agents such as WikiResearcher-9B to learn an effective search strategy.
3. A unified evaluation across all eight publicly released ViDoRe v3 splits(Loison et al. 2026), comparing WikiResearcher-9B with untrained baselines and larger external models under a common harness, together with out-of-domain evaluation on FinanceBench, LoCoMo, LongMemEval, and PersonaMem-v2 (Islam et al. 2023; Maharana et al. 2024; Wu et al. 2024; Jiang et al. 2025a).

The rest of the paper is organized as follows. Section 2 discusses related approaches. Section 3 presents the SearchWiki wiki representation, ingestion process, tool interface, and DAPO training recipe. Section 4 describes the experimental setup. Section 5 reports ViDoRe v3 results, ablations, generalization holdouts, and analyses when navigation helps. Section 6 concludes and summarizes the manuscript.

2 Related Work

Retrieval-augmented generation. RAG systems retrieve top-k passages and feed them to a generator. ViDoRe (Faysse et al. 2025; Loison et al. 2026) stresses retrieval over visually complex, multi-page documents. SearchWiki differs by treating retrieval as an interactive search-and-read loop over a pre-compiled wiki, rather than a single flat context pack.

Agentic search and reinforcement learning. Search-R1 (Jin et al. 2025) and DeepRetrieval (Jiang et al. 2025b) train agents to issue search queries. GrepSeek (Salemi et al. 2026) trains agents to operate on raw text with Unix-style tools. Interact-RAG (Hui et al. 2026) exposes fine-grained corpus operations as interactive primitives. KARL (Chang et al. 2026), DeepRAG (Guan et al. 2026) and GraphRAG-R1 (Yu et al. 2026a) further study RL for retrieval decisions. TMAX (Ivison et al. 2026) shows that simple terminal rewards can suffice in constrained agent settings. SearchWiki uses a different *search space*: a typed, structured wiki (document summaries, cross-document topics, and page-level sources) that the agent traverses with domain-agnostic tools, rather than a flat search API or an unstructured text dump.

Structured memory versus corpus wikis. Memory systems for personal assistants organize user histories as records, graphs, or hierarchies (Zhong et al. 2024; Packer et al. 2023; Chhikara et al. 2025). AgeMem (Yu et al. 2026b) and NapMem (Xu et al. 2026) train agents to navigate multi-layer *user memory*, with NapMem specializing its RL policy on memory benchmarks such as LoCoMo, LongMemEval, and PersonaMem-v2. Similarly, PageIndex (Zhang, Tang, and Team 2025), proposes vectorless RAG by constructing a hierarchical tree index from long documents; Inspired by LLM-wiki (Karpathy 2026), SearchWiki aims to convert a given *document corpora* to a domain-agnostic wiki format (Ming et al. 2026) and provides a harness with an effective tool set to traverse the wiki efficiently. Given the same wiki representation, a SearchWiki harness using both an untrained large as well as our ViDoRe-trained WikiResearcher-9B model outperform NapMem's reported memory-specialized numbers on key metrics on LoCoMo (and the harness also leads on PersonaMem-v2; Table 3), suggesting that structured navigation transfers beyond the training domain.

RL for tool-using language models. GRPO (Shao et al. 2024) estimates advantages from group-normalized rewards without a critic. DAPO (Yu et al. 2025) uses asymmetric clipping and dynamic sampling for long-trajectory stability. RTM-style masking (Jin et al. 2025) avoids updating the policy on environment tokens. We combine these ingredients with a SearchWiki-specific dense reward stack.

3 SearchWiki

3.1 Overview

SearchWiki is a two-stage framework for active information seeking over complex document corpora: (1) **Wiki Compilation**, which transforms an unstructured corpus into a

typed, navigable three-layer wiki; and (2) **Corpus Navigation**, where an RL-tuned agent actively navigates the wiki via multi-turn tool interaction. Figure 1 depicts this as three stages - wiki creation, harness-based inference, and the offline RL training loop (Section 3.3) that produces the navigation policy used at inference.

Given a user query q and a compiled SearchWiki knowledge base $\mathcal{K}$, the agent interacts with the environment over T steps to construct a decision trajectory:

$$\tau = (a_1, o_1, a_2, o_2, \ldots, a_T, o_T, y), \tag{1}$$

where a_t represents an assistant action (comprising chain-of-thought planning, scratchpad updates, or tool calls), o_t is the environmental observation returned by the tool execution engine and appended to the context history (Yao et al. 2023), and y is the terminal answer produced when search terminates. Rather than relying on single-shot passage retrieval, π_θ dynamically decides which nodes to inspect, when to descend into page-level details, and when to backtrack based on intermediate observations.

3.2 Wiki Compilation and Ingestion

SearchWiki wikis are built offline with a fully autonomous LLM compilation process. Figure 1 (left) summarizes the path from a raw knowledge source to the typed wiki layout.

Knowledge Representation. To preserve document hierarchy and cross-document relationships, SearchWiki organizes a corpus $\mathcal{K}$ into a local wiki containing four node types::

1. **Document overviews (*documents/*):** one front-matter-typed summary per document (e.g., a Form 10-K overview) that lists section structure with page ranges and links into section sub-folders such as *documents/doc_id/part_i/item_1_business.md*.
2. **Topics (*topics/*):** Cross-document concept pages (e.g., risk management policies) containing markdown links to related document overviews and cross-referenced topics.
3. **Source pages (*sources/*):** per-document JSON lists of page records (OCR/layout text, page index, and related metadata) loaded on demand via *read_page*.
4. **Catalog files:** root and per-directory *index.md* listings that map the tree, plus a build *log.md*.

Details about the directory layout has been added to the supplementary .

The agent is not given the full corpus at once. It discloses content through: search and catalog reads, inspect document or topic summaries, then call *read_page* for exact evidence.

This same wiki structure naturally extends to dialogue corpora by mapping session overviews to *documents/* and turn records to *sources/*. We train WikiResearcher-9B on document-grounded ViDoRe wikis and demonstrate out-of-distribution generalization to dialogue-grounded memory benchmarks.

Compilation Pipeline. The offline wiki construction process executes in three automated stages:

Stage 1: Layout-Aware Parsing. Document page images are processed with a vision-language OCR/layout pipeline to extract text and structure. Short text inputs skip heavy visual parsing. Parsed pages are serialized as page JSON records under *sources/*, enabling downstream tools to target exact page ranges without re-parsing raw files. Short text inputs skip visual parsing.

Stage 2: TOC Extraction and Progressive Structuring. For long documents, our hierarchical indexer extracts a table of contents (TOC) that records section titles, nesting depth, and page spans. That TOC is the blueprint for the on-disk tree under *documents/*: the document root summary links to top-level sections; each section becomes a subdirectory with its own *index.md* and section summary; nested TOC entries spawn further subdirectories and subsection summaries (sections within sections), each tied to the corresponding source page range. Short documents without a usable TOC take a direct single-summary route. Conversational memory sources follow the same principle at a different granularity: each conversation is compiled session by session, with every session becoming a short document and each turn a linked subsection tied back to the raw dialogue under *sources/*. The resulting hierarchy acts as a condensed information proxy: the agent can traverse Part →Item→subsection summaries before calling *read_page* on the raw sources.

Stage 3: Joint Wiki Compiler. For each document unit, a single LLM call simultaneously generates (i) TOC-aligned document and section summaries and (ii) concept registration actions against a global topic registry. Performing summary generation and concept planning jointly minimizes information loss common in sequential pipelines and prevents duplicate topic creation. Compilation across independent documents is parallelized, while chronological sources are processed sequentially to maintain a shared topic registry. Prompt constraints encourage canonical topic names and discourage near-duplicate concepts.

Tool	Purpose
search_wiki	Pipe-split multi-query BM25 with RRF; returns top documents and page hints
grep_search	Regex scan over documents, topics, and sources
read_file	Read markdown summaries, sections, or topic pages
read_page	Read page-level source JSON for selected pages
list_files	List files in a directory
glob_file_search	Match filenames by pattern
update_scratchpad	Search planning: persistent notes on sub-questions and progress

Table 1: SearchWiki tool suite and purpose used in training and evaluation.

3.3 Wiki Navigation and Policy Learning

Tool-Based Navigation. We list seven tools available to the agent in Table 1. The system prompt requires *up-*

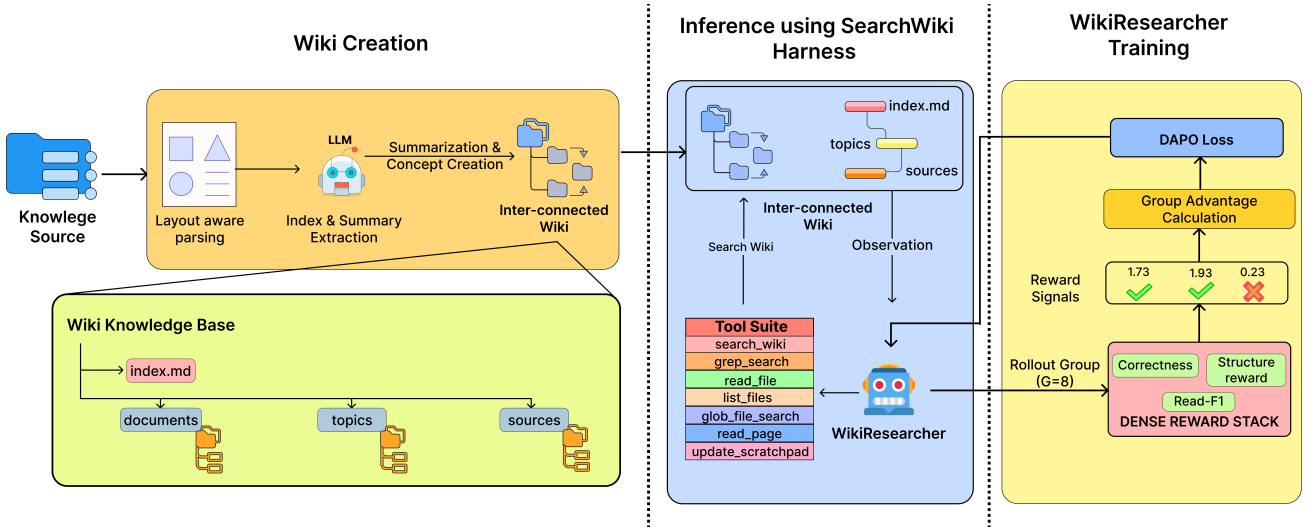


Figure 1: SearchWiki end-to-end framework. Left: layout-aware parsing, TOC extraction, and joint summarization compile an unstructured knowledge source into a typed, three-layer wiki (*documents/*, *topics/*, *sources/*). Middle: given a query, the SearchWiki harness navigates the wiki through a multi-turn tool-use loop (search, read, verify), returning a cite-tagged final answer. Right: WikiResearcher-9B is trained on rollout groups scored by the dense reward stack and optimized via DAPO.

date_scratchpad on every tool turn, in parallel with other calls for persistent planning.

Output protocol. Each assistant turn follows a Qwen3.5-style XML protocol (Qwen Team 2026): an optional <*think*> block for reasoning, zero or more <*tool_call*> blocks using the native <*function*=...> parameter format, or a terminal <*answer*> block. Final answers must conclude with explicit citations in the form *[Relevant Documents: DOC_NAME[pages]]*. Trajectory format compliance is validated via automated tag checking, with malformed outputs down-weighted during training. Full multi-turn trajectory examples are included in the supplementary material.

Policy Optimization via Reinforcement Learning. We train the navigation policy π_θ online: for each training query, the agent generates multi-turn tool-use trajectories against the corresponding wiki environment. Our RL alignment strategy rests on the following core components:

Online rollouts. Each step samples training queries and generates $G = 8$ completions per prompt. A multi-turn scheduler executes tool calls against the split-local wiki and appends observations to the context dialogue.

Retrieved Token Masking. Tool responses o_t are included in the context history but excluded from policy loss calculation (Jin et al. 2025). Gradient updates are restricted exclusively to model-generated tokens (planning, tool calls, and answers), preventing the policy from imitating retrieved OCR text.

DAPO objective. For each query we sample a group of G trajectories with scalar rewards $\{R_i\}_{i=1}^{G}$ and form group-relative advantages

$$\hat{A}_i = \frac{R_i - \mathrm{mean}(\{R_j\}_{j=1}^{G})}{\mathrm{std}(\{R_j\}_{j=1}^{G}) + \delta}, \qquad \delta = 10^{-8}. \tag{2}$$

Following DAPO (Yu et al. 2025), we maximize the token-level clipped surrogate with asymmetric clip bounds

$$J_{\mathrm{DAPO}}(\theta) = \mathbb{E}\left[\frac{1}{Z}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\big(r_{i,t}(\theta)\,\hat{A}_i,\ \mathrm{clip}\big(r_{i,t}(\theta),\,1-\varepsilon_{\mathrm{low}},\,1+\varepsilon_{\mathrm{high}}\big)\,\hat{A}_i\big)\right], \tag{3}$$

where $Z = \sum_{i=1}^{G}|o_i|$, the expectation is over $(q, \{o_i\}) \sim \mathcal{D}$, and the importance ratio is

$$r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\mathrm{old}}}(o_{i,t} \mid q, o_{i,<t})}. \tag{4}$$

We set $\varepsilon_{\mathrm{low}} = 0.2$ and $\varepsilon_{\mathrm{high}} = 0.28$ (Clip-Higher). Because Retrieved Token Masking zeros the contribution of tool-observation tokens, the inner sum runs only over model-generated tokens (planning, tool calls, and answers). We use DAPO for the policy update and GDPO (Liu et al. 2026) for group reward scaling under the same reward stack.

Two-stage training. Training proceeds in two phases. In Stage 1, the policy is optimized end-to-end using the full search and QA reward stack. In Stage 2, we restrict gradient updates strictly to final answer tokens while retaining the QA correctness reward. Training with QA rewards alone without this token mask collapses navigation policy learning; this two-stage, answer-masked recipe prevents trajectory degradation over long search chains.

Dense reward stack. The scalar reward combines answer quality, retrieval quality, and trajectory discipline in a weighted sum formulation:

- **Correctness**: LLM as a Judge for scoring correctness - 1.0 for Correct, 0.3 for partial and 0 for incorrect (Zheng et al. 2023; Liu et al. 2023).
- **Read-F1** and **retrieval rank / NDCG**: reward reading and ranking gold pages/documents by citation and reads (Järvelin and Kekäläinen 2002).
- **Structural Rewards:** format compliance, discovery-phase traversal heuristics, search-tool usage, and parallel-tool bonuses; plus penalties for excess turns, duplicate calls, answering without tools, or non-submitting finals.

Exact component weights have been listed in the supplementary material. The reward mixture was calibrated through analysis of failure modes: correctness alone risks reward guessing, retrieval alone incentivizes reading without answering, and structural shaping alone encourages busywork (Ng, Harada, and Russell 1999; Amodei et al. 2016; Skalse et al. 2022). Rollouts truncated by context limits or turn bounds receive a hard total reward of zero ($R = 0$), enforcing budget discipline during exploration.

4 Experimental Setup

4.1 Training Setup

Synthetic Data Generation. To train our RL policy, we utilize a three-stage synthetic data generation pipeline inspired by ViDoRe. First, for each gold evidence unit, we prompt a teacher LLM to generate candidate questions grounded in the underlying selected source text. After generation, candidates are passed through a deterministic filter, which includes regular expressions, length validation, coverage verification and checks to eliminate trivial queries containing direct answer leakage. Following this, we evaluate the remaining queries by an LLM judge across four critical rubrics: factual faithfulness, query clarity, absence of answer leakage, and adherence to formatting style. A candidate is discarded if it fails to follow any criterion of the four. This filtering resulted in retaining approximately 25% of the total generated queries, yielding a final dataset of $1{,}800$ high quality instances. Each domain split contains 225 queries, and we divide our data into $1{,}600$ training samples and 200 validation samples.

Implementation Details. We initialize WikiResearcher-9B from Qwen3.5-9B and train with DeepSpeed ZeRO-3 (Rajbhandari et al. 2020), sequence parallelism of 4 across 8 A100 GPUs, and a colocated vLLM engine for generation, optimizing over $1{,}600$ synthetic queries for one epoch ($\approx$400 steps) at a peak learning rate of 10^{-6}. Full training configuration is in the supplementary material.

Context Budget and Reasoning Length. Long CoT reasoning inflates context usage across multi-turn trajectories, pushing rollouts toward the $65{,}536$-token limit; since truncated trajectories receive zero reward, verbose thinking can destabilize training despite appearing high-quality. We therefore favor concise step-by-step planning paired with the scratchpad tool over unconstrained reasoning traces.

4.2 Evaluation Framework

We evaluate performance along two primary axes: answer quality, assessed via an LLM-as-judge protocol, and retrieval quality, measured through page- and document-level ranking metrics. Crucially, all evaluations execute under the identical agent scheduler, tool suite, and environment configuration used during training. This guarantees that observed gains reflect genuine improvements in the learned navigation policy rather than artifacts of a mismatched test harness.

Datasets and Benchmarks. We evaluate in-distribution performance on all eight *ViDoRe v3* domains. For out-of-distribution transfer, we evaluate on *FinanceBench* (financial QA) and three dialogue memory benchmarks: *LoCoMo*, *LongMemEval*, and *PersonaMem-v2* (Loison et al. 2026; Islam et al. 2023; Maharana et al. 2024; Wu et al. 2024; Jiang et al. 2025a). While evaluating on memory benchmarks, we adopt NapMem's exact setup (user-disjoint $60/20/20$ splits, token F1, LLM-judge, and multiple-choice accuracy) for direct comparability (Xu et al. 2026).

Answer Correctness Metrics. Answer quality is evaluated using GPT-OSS-120B (OpenAI 2025) as an external LLM judge. For *ViDoRe v3*, we adopt the official ViDoRe judge prompt (Loison et al. 2026), which categorizes answers as Correct, Partially Correct, or Incorrect. For out-of-distribution holdouts, we use NapMem's binary Correct/Incorrect judge prompt (or multiple-choice accuracy where applicable). Across all open-ended tasks, we report strict accuracy—counting only fully Correct answers—both per split and macro-averaged across domains. Full judge templates and prompts are included in the supplementary material.

Retrieval and Ranking Metrics. Retrieval quality uses Recall@5 and nDCG@5 at page and document levels. Rankings follow a cite-then-read convention: cited evidence ranks first, followed by pages/documents accessed via read_page or read_file. Page-level metrics may understate performance when agents answer from summaries or topic pages without accessing the exact source; document-level metrics are more robust to this.

Comparative Baselines. All evaluated models operate under the identical SearchWiki tool suite and environment harness. We compare our RL-tuned policy, WikiResearcher-9B, against its same-size untrained base model (Qwen3.5-9B) as well as several larger baseline models: Qwen3.6-27B, Qwen3.5-397B, Gemma-4-31B, and MiniMax-M2.5 (Qwen Team 2026; Gemma Team 2026; MiniMax 2026).

5 Results and Analysis

5.1 Results on ViDoRe v3

WikiResearcher-9B outperforms other baselines on end-to-end QA accuracy: We report the per-split QA accuracy under the official ViDoRe LLM-as-judge prompt (Loison et al. 2026) (judge model: GPT-OSS-120B) in Table 2. WikiResearcher-9B leads at 71.35 ± 0.73, ahead of untrained Qwen3.6-27B (70.94 ± 0.72) and Qwen3.5-397B (69.51 ± 0.73), and improves $+7.4$ points over the same-size

Model	English Datasets					French Datasets			Avg.
	C.S.	Fin.	Phar.	H.R.	Ind.	Phys.	Ener.	Fin.	
Gemma-4-31B†	81.47	55.45	60.83	56.39	52.36	74.94	58.82	51.46	61.47±0.79
MiniMax-M2.5†	71.63	20.23	35.16	24.11	23.97	70.42	29.33	23.28	37.27±0.72
Qwen3.5-397B†	86.67	63.54	72.39	64.41	58.78	83.39	69.16	57.76	69.51±0.73
Qwen3.6-27B†	87.36	65.43	71.52	68.13	61.84	82.67	71.05	59.48	70.94±0.72
Qwen3.5-9B†	82.33	57.61	67.81	54.72	51.83	79.69	63.31	54.32	63.95±0.76
WikiResearcher-9B	**86.51**	**65.80**	**77.84**	**63.94**	**61.01**	**83.94**	**74.03**	**57.71**	**71.35**±0.73

Table 2: ViDoRe v3 strict QA accuracy (%; same SearchWiki harness). Avg. is the unweighted macro average over eight splits; ± is the 95% CI half-width ($n_{\text{boot}}=10^4$; per-split results in supplementary); †=untrained. WikiResearcher-9B is bold; best per column is underlined.

Method	LoCoMo		LongMemEval		PersonaMem-v2
	F1	LLM-J	F1	LLM-J	Acc.
Zep‡	36.09±0.38	52.21±0.31	52.61±0.20	73.33±0.94	36.33±0.81
MemoryOS‡	31.40±0.53	44.28±1.22	22.68±0.30	23.67±0.58	35.82±1.87
MemOS‡	35.22±0.38	55.82±0.65	39.16±0.40	54.33±0.47	37.58±0.44
AgeMem‡	38.00±0.12	45.02±0.12	37.83±1.38	51.33±0.94	24.19±0.67
Mem0‡	41.19±0.36	60.86±0.38	53.86±0.54	78.00±0.82	38.89±0.34
NapMem-397B (Untrained)‡	38.31±0.13	54.42±0.81	53.85±1.25	78.33±2.08	46.10±1.15
NapMem-9B w/ RL‡	41.28±0.59	59.92±0.78	57.41±1.12	80.33±0.47	47.97±1.01
Gemma-4-31B†	48.04±0.50	58.99±0.13	54.45±0.15	67.50±2.50	46.87±0.66
MiniMax-M2.5†	24.60±1.23	33.28±1.51	22.41±2.76	30.50±2.50	34.36±0.33
Qwen3.5-397B†	52.10±0.16	63.87±0.23	56.22±0.80	69.50±0.50	50.00±0.82
Qwen3.6-27B†	55.84±1.65	69.03±1.89	61.70±2.58	77.50±3.50	57.08±3.18
WikiResearcher-9B	**48.31±0.18**	**63.65±0.10**	**59.81±1.46**	**80.67±2.50**	**47.26±0.05**

Table 3: Memory-intensive task performance on LoCoMo, LongMemEval, and PersonaMem-v2 (%; mean±uncertainty). (‡=results from NapMem (Xu et al. 2026)); (†=untrained). Uncertainty details in the supplementary. WikiResearcher-9B is bold; best per column is underlined.

Qwen3.5-9B (63.95±0.76). Paired bootstrap comparisons put the 9B and 397B gains well clear of zero; the edge over 27B is smaller but still favors WikiResearcher-9B (details in Supplementary material).

WikiResearcher-9B leads retrieval metrics across domains. We summarize the cite-then-read Recall@5 and nDCG@5 at page and document level (macro average over splits) in Table 4 . WikiResearcher-9B improves page-level nDCG@5 over both untrained models, consistent with our retrieval-aligned reward stack, and remains competitive at the document level – notably surpassing Qwen3.5-397B, a same-family model with 44× the parameters, on page-level recall and nDCG. Per-domain breakdowns are in the supplementary material.

Model	Page		Doc	
	nDCG@5	R@5	nDCG@5	R@5
Gemma-4-31B†	43.82	38.16	82.89	85.02
MiniMax-M2.5†	20.84	18.35	48.14	50.09
Qwen3.5-397B†	47.15	43.21	87.66	91.33
Qwen3.6-27B†	47.05	43.70	87.41	90.41
Qwen3.5-9B†	40.55	37.16	84.23	89.35
WikiResearcher-9B	**53.22**	**48.55**	**86.83**	**90.02**

Table 4: ViDoRe v3 cite-then-read retrieval (%), averaged over splits. Same models/order/harness as Table 2; †=untrained. WikiResearcher-9B is bold; best per column is underlined.

5.2 Performance on Memory Benchmarks

We report LoCoMo, LongMemEval, and PersonaMem-v2 under NapMem's evaluation protocol in Table 3 (with same 60/20/20 user-level splits, judge prompt, and F1 / LLM-J metrics), so gaps vs. NapMem are not an artifact of mismatched splits or rubrics. NapMem's strongest reported 9B agent is trained specifically for user-memory navigation on these suites (Xu et al. 2026). SearchWiki is not: harness rows reuse the same wiki tools as ViDoRe, and WikiResearcher-9B is the ViDoRe-trained 9B student. Still, SearchWiki leads NapMem on LoCoMo F1 and LLM-judge (27B harness), and WikiResearcher-9B is competitive on LongMemEval LLM-J (80.67 vs. NapMem's 80.33) despite no memory-specific training. The untrained 27B harness also exceeds NapMem on PersonaMem-v2, indicating that a general wiki navigator can match or beat a memory-specialized policy without memory-specific training.

5.3 Generalization to FinanceBench

We report the results on FinanceBench dataset in Table 5. WikiResearcher-9B reaches 83.33% accuracy, well above the leaderboard baselines from Islam et al. (2023) (GPT-4-Turbo, GPT-4, Llama-2, all $\leq$19.30%) and close to the GPT-4-Turbo *Oracle* upper bound (85.33%), despite using retrieved rather than gold evidence. Within our own harness, where all models share identical retrieval, WikiResearcher-9B trails the best system, Qwen3.6-27B† (85.33%), by only 2 points, and Qwen3.5-397B (84.67%) by 1.34 – notable given WikiResearcher-9B is trained solely on ViDoRe. This out-of-distribution transfer suggests its evidence-localization and wiki-traversal skills generalize well beyond the training domain.

Model / Setting (*Shared Store*)	**Acc. (%)**
GPT-4-Turbo(*Oracle*)‡	85.33
GPT-4-Turbo‡	19.30
Llama-2‡	19.30
GPT-4‡	16.70
Gemma-4-31B†	78.00
MiniMax-M2.5†	29.33
Qwen3.5-397B†	84.67
Qwen3.5-9B†	79.33
Qwen3.6-27B†	<u>85.33</u>
WikiResearcher-9B	**83.33**

Table 5: FinanceBench Shared-Store results (%). (‡ numbers from (Islam et al. 2023).) Bottom block: same SearchWiki harness and model order as Table 2.†=untrained. WikiResearcher-9B is bold; best in the harness block is underlined.

5.4 Ablations

We disentangle the contributions of our hierarchical wiki representation and multi-turn tool use in Table 6, while comparing against passive single-turn RAG baselines.

We consider two ablations of SearchWiki. (1) *w/o typed hierarchy* retains the full tool suite but exposes the agent only to a flat collection of source pages, removing the topic and document layers from the wiki. (2) *w/o tools (BM25)* removes multi-turn interaction entirely, instead retrieving BM25 top-k OCR chunks and generating an answer in a single pass. We additionally report a stronger passive baseline, *dense RAG*, which retrieves the top-10 pages using *nvidia/nemotron-colembed-vl-8b-v2* and answers in one turn with the same untrained 9B/27B generators.

The results show consistent gains from both hierarchy and interaction. With the untrained 27B model, full SearchWiki achieves 70.94, outperforming the flat-hierarchy agent (65.47), dense RAG (62.61), and BM25 RAG (29.60). The same trend holds with the untrained 9B model, where SearchWiki reaches 63.95, compared to 57.30 for dense RAG and 50.39 without the typed hierarchy. Applying WikiResearcher-9B further improves the 9B model to 71.35, demonstrating that reinforcement learning provides gains beyond model scaling, while the hierarchical representation and multi-turn interaction each contribute independently beyond simply replacing BM25 with a stronger dense retriever.

Setting	**Avg. QA**
Full SearchWiki (27B)	70.94
w/o typed hierarchy	65.47
dense RAG (Dense, single-turn)	62.61
w/o tools (BM25, single-turn)	29.60
Full SearchWiki (9B, w/o RL)	63.95
w/o typed hierarchy	50.39
dense RAG (Dense, single-turn)	57.30
WikiResearcher-9B	**<u>71.35</u>**

Table 6: ViDoRe v3 ablations (macro-average QA %; judge: GPT-OSS-120B). Dense RAG uses *nvidia/nemotron-colembed-vl-8b-v2* (top-10). †=untrained. WikiResearcher-9B is bold; best score is underlined.

These improvements stem from the different search paradigm enabled by SearchWiki. In passive RAG, recovery from an initial retrieval miss is largely restricted to reformulating retrieval queries while continuing to interact blindly with the corpus. In contrast, SearchWiki treats recovery as an explicit decision process: the agent can iteratively revise queries, inspect summaries, traverse topic links, search for literals with *grep*, and finally verify evidence using *read_page*. The typed hierarchy is central to this process, as topic and summary pages provide inexpensive high-level context that guides exploration before the agent commits to reading source pages. Rather than serving solely as an improved retrieval index, the wiki becomes a navigable action space that supports efficient information seeking.

6 Conclusion

In this paper, we introduced SearchWiki, a two-stage framework that replaces passive flat RAG with active corpus navigation. First, SearchWiki automatically compiles unstructured corpora into a typed, three-layer wiki comprising document overviews, cross-document topic pages, and page-level source records. Second, we train WikiResearcher-9B via online RL to master multi-turn tool use, evidence verification, and dynamic backtracking. Our empirical results strongly validate this paradigm: WikiResearcher-9B ranks first overall on ViDoRe v3 in end-to-end QA accuracy (71.35%)—outperforming models up to 44$\times$ its parameter scale - while dominating page-level retrieval ranking (53.22% nDCG@5). On FinanceBench, WikiResearcher-9B achieves 83.33% accuracy - approaching the Oracle upper bound (85.33%). Finally, it surpasses domain-specialized memory policies on LoCoMo, PersonaMem-v2 and LongMemEval without any memory-specific training, establishing learned wiki navigation as a superior and generalizable alternative to chunk-based retrieval.

References

Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; and Hajishirzi, H. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In Kim, B.; Yue, Y.; Chaudhuri, S.; Fragkiadaki, K.; Khan, M.; and Sun, Y., eds., *International Conference on Learning Representations*, volume 2024, 9112–9141.

Bates, M. J. 1989. The Design of Browsing and Berrypicking Techniques for the Online Search Interface. *Online Review*, 13(5): 407–424.

Chang, J. D.; Drozdov, A.; Toshniwal, S.; Oertell, O.; Trott, A.; Portes, J.; Gupta, A.; Koppol, P.; Baheti, A.; Kulinski, S.; Zhou, I.; Dea, I.; Opsahl-Ong, K.; Favreau-Lessard, S.; Owen, S.; Gonzalez Ortiz, J. J.; Singhvi, A.; Andrade, X.; Wang, C.; Sreenivasan, K.; Havens, S.; Liu, J.; DeNiro, P.; Sun, W.; Bendersky, M.; and Frankle, J. 2026. KARL: Knowledge Agents via Reinforcement Learning. arXiv:2603.05218.

Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In Lynce, I.; Murano, N.; Vallati, M.; Villata, S.; Chesani, F.; Milano, M.; Omicini, A.; and Dastani, M., eds., *ECAI 2025 – 28th European Conference on Artificial Intelligence, 25–30 October 2025, Bologna, Italy – Including 14th Conference on Prestigious Applications of Intelligent Systems (PAIS 2025)*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, 2993–3000. IOS Press.

Dai, J.; Wu, D.; Chen, Y.; Zeng, Z.; Yan, Y.; Liu, Z.; and Sun, M. 2026. NaviRAG: Towards Active Knowledge Navigation for Retrieval-Augmented Generation. *arXiv preprint arXiv:2604.12766*.

Du, M.; Xu, B.; Zhu, C.; Wang, S.; Wang, P.; Wang, X.; and Mao, Z. 2026. A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces. *arXiv preprint arXiv:2602.03442*.

Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R. O.; and Larson, J. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*.

Faysse, M.; Sibille, H.; Wu, T.; Omrani, B.; Viaud, G.; Hudelot, C.; and Colombo, P. 2025. ColPali: Efficient Document Retrieval with Vision Language Models. In *The Thirteenth International Conference on Learning Representations*. Singapore: OpenReview.net.

Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, M.; and Wang, H. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.

Gemma Team. 2026. Gemma 4 Technical Report. arXiv:2607.02770.

Guan, X.; Zeng, J.; Meng, F.; Xin, C.; Lu, Y.; Lin, H.; Han, X.; Sun, L.; and Zhou, J. 2026. DeepRAG: Thinking to Retrieve Step by Step for Large Language Models. In *The Fourteenth International Conference on Learning Representations*. OpenReview.net.

Gutiérrez, B. J.; Shu, Y.; Gu, Y.; Yasunaga, M.; and Su, Y. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J.; and Zhang, C., eds., *Advances in Neural Information Processing Systems*, volume 37, 59532–59569. Curran Associates, Inc.

Hui, Y.; Chen, C.; Fu, Z.; Liu, Y.; Ye, J.; and Zhang, H. 2026. Interact-RAG: Reason and Interact with the Corpus, Beyond Black-Box Retrieval. In *The Fourteenth International Conference on Learning Representations*. OpenReview.net.

Islam, P.; Kannappan, A.; Kiela, D.; Qian, R.; Scherrer, N.; and Vidgen, B. 2023. FinanceBench: A New Benchmark for Financial Question Answering. arXiv:2311.11944.

Ivison, H.; Yin, J. O.; Shao, R.; Xiao, T.; Lambert, N.; and Hajishirzi, H. 2026. Tmax: A simple recipe for terminal agents. arXiv:2606.23321.

Järvelin, K.; and Kekäläinen, J. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4): 422–446.

Jeong, S.; Baek, J.; Cho, S.; Hwang, S. J.; and Park, J. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 7036–7050. Mexico City, Mexico: Association for Computational Linguistics.

Jiang, B.; Yuan, Y.; Shen, M.; Hao, Z.; Xu, Z.; Chen, Z.; Liu, Z.; Vijjini, A. R.; He, J.; Yu, H.; Poovendran, R.; Wornell, G.; Ungar, L.; Roth, D.; Chen, S.; and Taylor, C. J. 2025a. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory. *arXiv preprint arXiv:2512.06688*.

Jiang, P.; Lin, J.; Cao, L.; Tian, R.; Kang, S.; Wang, Z.; Sun, J.; and Han, J. 2025b. DeepRetrieval: Hacking Real Search Engines and Retrievers with Large Language Models via Reinforcement Learning. In *The Second Conference on Language Modeling*.

Jiang, Z.; Xu, F.; Gao, L.; Sun, Z.; Liu, Q.; Dwivedi-Yu, J.; Yang, Y.; Callan, J.; and Neubig, G. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 7969–7992. Singapore: Association for Computational Linguistics.

Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arık, S. Ö.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In *The Second Conference on Language Modeling*.

Karpathy, A. 2026. LLM Wiki. GitHub Gist. A pattern for building personal knowledge bases using LLMs; non-archival idea file created 2026-04-04; accessed 2026-07-28.

Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; and Yih, W.-t. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In Webber, B.; Cohn, T.; He, Y.; and Liu, Y., eds., *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781. Online: Association for Computational Linguistics.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.-F.; and Lin, H.-T., eds., *Advances in Neural Information Processing Systems*, volume 33, 9459–9474. Curran Associates, Inc.

Liu, S.-Y.; Dong, X.; Lu, X.; Diao, S.; Belcak, P.; Liu, M.; Chen, M.-H.; Yin, H.; Wang, Y.-C. F.; Cheng, K.-T.; Choi, Y.; Kautz, J.; and Molchanov, P. 2026. GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization. arXiv:2601.05242.

Liu, Y.; Hashimoto, K.; Zhou, Y.; Yavuz, S.; Xiong, C.; and Yu, P. 2021. Dense Hierarchical Retrieval for Open-domain Question Answering. In Moens, M.-F.; Huang, X.; Specia, L.; and Yih, S. W.-t., eds., *Findings of the Association for Computational Linguistics: EMNLP 2021*, 188–200. Punta Cana, Dominican Republic: Association for Computational Linguistics.

Liu, Y.; Iter, D.; Xu, Y.; Wang, S.; Xu, R.; and Zhu, C. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2511–2522. Singapore: Association for Computational Linguistics.

Loison, A.; Macé, Q.; Edy, A.; Xing, V.; Balough, T.; Moreira, G. d. S. P.; Liu, B.; Faysse, M.; Hudelot, C.; and Viaud, G. 2026. ViDoRe V3: A Comprehensive Evaluation of Retrieval Augmented Generation in Complex Real-World Scenarios. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 16570–16600. San Diego, California, United States: Association for Computational Linguistics.

Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In Ku, L.-W.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13851–13870. Bangkok, Thailand: Association for Computational Linguistics.

Ming, H.; Li, F.; Wu, X.; and Que, W. 2026. Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki. *arXiv preprint arXiv:2605.25480*.

MiniMax. 2026. MiniMax M2.5: Built for Real-World Productivity. https://www.minimax.io/news/minimax-m25. Published: 2026-02-12; Accessed: 2026-07-28.

Ng, A. Y.; Harada, D.; and Russell, S. J. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, 278–287. San Francisco, CA, USA: Morgan Kaufmann. ISBN 1-55860-612-2.

OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925.

Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.

Pirolli, P.; and Card, S. 1999. Information Foraging. *Psychological Review*, 106(4): 643–675.

Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. Accessed: 2026-07-12.

Rajbhandari, S.; Rasley, J.; Ruwase, O.; and He, Y. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–16. IEEE.

Salemi, A.; Zeng, C.; Nijasure, A.; Chung, J.-H.; Rahimi, R.; Diaz, F.; and Zamani, H. 2026. GrepSeek: Training Search Agents for Direct Corpus Interaction. *arXiv preprint arXiv:2605.29307*.

Sarthi, P.; Abdullah, S.; Tuli, A.; Khanna, S.; Goldie, A.; and Manning, C. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In Kim, B.; Yue, Y.; Chaudhuri, S.; Fragkiadaki, K.; Khan, M.; and Sun, Y., eds., *International Conference on Learning Representations*, volume 2024, 32628–32649.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Shuster, K.; Poff, S.; Chen, M.; Kiela, D.; and Weston, J. 2021. Retrieval Augmentation Reduces Hallucination in Conversation. In Moens, M.-F.; Huang, X.; Specia, L.; and Yih, S. W.-t., eds., *Findings of the Association for Computational Linguistics: EMNLP 2021*, 3784–3803. Punta Cana, Dominican Republic: Association for Computational Linguistics.

Skalse, J.; Howe, N. H. R.; Krasheninnikov, D.; and Krueger, D. 2022. Defining and Characterizing Reward Gaming. In *Advances in Neural Information Processing Systems*, volume 35, 9460–9471.

Sun, Y.; Wei, P.; and Hsieh, L. B. 2026. Don't Retrieve, Navigate: Distilling Enterprise Knowledge into Navigable Agent Skills for QA and RAG. *arXiv preprint arXiv:2604.14572*.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 10014–10037. Toronto, Canada: Association for Computational Linguistics.

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.

Xu, Y.; Sun, Y.; Liu, Y.; Zhou, M.; Qiao, J.; Ma, L.; Tang, K.; Wang, W.; Jiang, X.; and Jiang, G. 2026. From Passive Retrieval to Active Memory Navigation: Learning to Use Memory as a Structured Action Space. arXiv:2607.05794.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Yu, C.; Zhao, K.; Li, Y.; Chang, H.; Feng, M.; Jiang, X.; Sun, Y.; Li, J.; Zhang, Y.; Sun, Q.; Li, J.; and Zhang, Z. 2026a. GraphRAG-R1: Graph Retrieval-Augmented Generation with Process-Constrained Reinforcement Learning. In *Proceedings of the ACM Web Conference 2026*, 1398–1409. New York, NY, USA: Association for Computing Machinery.

Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, J.; Liu, L.; Liu, X.; Lin, H.; Lin, Z.; Ma, B.; Sheng, G.; Tong, Y.; Zhang, C.; Zhang, M.; Zhang, R.; Zhang, W.; Zhu, H.; Zhu, J.; Chen, J.; Chen, J.; Wang, C.; Yu, H.; Song, Y.; Wei, X.; Zhou, H.; Liu, J.; Ma, W.-Y.; Zhang, Y.-Q.; Yan, L.; Wu, Y.; and Wang, M. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In Belgrave, D.; Zhang, C.; Lin, H.; Pascanu, R.; Koniusz, P.; Ghassemi, M.; and Chen, N., eds., *Advances in Neural Information Processing Systems*, volume 38, 113222–113244. Curran Associates, Inc.

Yu, Y.; Yao, L.; Xie, Y.; Tan, Q.; Feng, J.; Li, Y.; and Wu, L. 2026b. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 21457–21483. San Diego, California, United States: Association for Computational Linguistics.

Zhang, M.; Tang, Y.; and Team, P. 2025. PageIndex: Next-Generation Vectorless, Reasoning-based RAG. *PageIndex Blog*. Https://pageindex.ai/blog/pageindex-intro.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 46595–46623.

Zhong, W.; Guo, L.; Gao, Q.; Ye, H.; and Wang, Y. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17): 19724–19731.